OBJ

Software Engineering Software Requirements Specification (SRS) Document

Tune Tribe

https://github.com/etgalang/TuneTribe

5/1/2024

1.0

By: Mauricio, Shauna, Emmanuel, Kannan

1. Introduction 3
1.1. Purpose 3
1.2. Document Conventions 3
1.3. Definitions, Acronyms, and Abbreviations 3
1.4. Intended Audience 4
1.5. Project Scope 5
1.6. Technology Challenges 5
1.7. References 6
2. General Description 7
2.1. Product Features 7
2.2. User Class and Characteristics 7
2.3. Operating Environment 8
2.4. Constraints 8
2.5. Assumptions and Dependencies 8
3. Functional Requirements 10
3.1. Primary 10
3.2. Secondary 10
3.3 Use-Case Model 11
3.3.1 Use-Case Model Diagram 11
3.3.2 Use-Case Model Descriptions 11
3.3.2.1 Actor: User (Mauricio) 11
3.3.2.2 Actor: Admin (Shauna) 12
3.3.2.3 Actor: Artist (Kannan) 12
3.3.3 Use-Case Model Scenarios 13
3.3.3.3 Actor: User (Mauricio) 13
3.3.3.4 Actor: Moderator (Emmanuel) 13
3.3.3.5 Actor: Artist (Kannan) 14
3.3.3.6 Actor: Admin (Shauna) 16
4 Technical Requirements 18
4.3 Interface Requirements 18
4.3.2 User Interfaces 18
4.3.3 Hardware Interfaces 19
4.3.4 Communications Interfaces 19
4.3.5 Software Interfaces 19
5 Non-Functional Requirements 20

5.3 Performance Requirements 20
5.4 Safety Requirements 20
5.5 Security Requirements 20
5.6 Software Quality Attributes 20
5.6.2 Availability 20
5.6.3 Correctness 20
5.6.4 Maintainability 20
5.6.5 Reusability 20
5.6.6 Portability 20
5.7 Process Requirements 21
5.7.2 Development Process Used 21
5.7.3 Time Constraints 21
5.7.4 Cost and Delivery Date 21
5.8 Other Requirements 21
6 Design Documents 21
6.3 Software Architecture 21
6.4 High-Level Database Schema 22
6.5 Software Design 23
6.5.2 State Machine Diagram: User (Mauricio) 23
6.5.3 State Machine Diagram: Shauna Ayscue (Admin) 23
6.5.4 State Machine Diagram: Emmanuel (Mod) 24
6.5.5 State Machine Diagram: Artist (Kennan) 24
6.6 UML Class Diagram Mod – Emmanuel(?) 25
7 Scenario 27
7.3 Brief Written Scenario with Screenshots 27

1. Introduction

1.1. Purpose (Mauricio)

TuneTribe is a social platform designed for music enthusiasts and artists alike. Users can share their favorite songs, create music-related posts, comment on friends' posts, and recommend songs/artists to each other. Artists have a dedicated space to upload their music, gain a following, and track royalties. The platform fosters community interaction while providing exposure for emerging artists. Moderators ensure a positive user experience by managing comments and addressing any issues. Admins oversee the platform's operations and have the authority to ban users or artists if necessary.

1.2. Document Conventions (Kannan)

The purpose of this Software Requirements Document (SRD) is to describe the client-view and developer-view requirements for the TuneTribe social web app. Client-oriented requirements describe the system from the client's perspective. These requirements include a description of the different types of users served by the system. Developer-oriented requirements describe the system from a software developer's perspective. These requirements include a detailed description of functional, data, performance, and other important requirements.

1.3. Definitions, Acronyms, and Abbreviations (Mauricio)

Java	A programming language originally developed by James Gosling at Sun Microsystems. We will be using this language to build the Restaurant Manager.
CSS	CSS stands for Cascading Style Sheets, and it's a style sheet language used for describing the presentation of a document written in a markup language like HTML.
JavaScript	JavaScript is a high-level, interpreted programming language used primarily to add interactivity and dynamic behavior to web pages.
MySQL	Open-source relational database management system.
.HTML	Hypertext Markup Language. This is the code that will be used to structure and design the web application and its content.
SpringBoot	An open-source Java-based framework used to create a micro Service. This will be used to create and run our application.
MVC	Model-View-Controller. This is the architectural pattern that will be used to implement our system.
Spring Web	Will be used to build our web application by using Spring MVC. This is one of the dependencies of our system.
Thymeleaf	A modern server-side Java template engine for our web environment. This is one of the dependencies of our system.
NetBeans	An integrated development environment (IDE) for Java. This is where our system will be created.

API	Application Programming Interface. This will be used to implement a function within the software where the current date and time is displayed on the homepage.

1.4. Intended Audience(???)

Intended Audience:

1. Stakeholders:
   - Investors or funding agencies
   - Executives or decision-makers within the organization
   - Legal advisors or regulatory bodies involved in compliance matters

   Relevant Sections: Entire SRS document for understanding project objectives, scope, and compliance requirements.

2. Developers:
   - Architects
   - Software Engineers
   - Frontend and Backend Developers

   Relevant Sections:
   - System architecture and design details
   - Technical specifications for frontend and backend components
   - API documentation
   - Database schema and data models
   - Coding standards and guidelines

3. **Project Managers**:
   - Project Managers
   - Team Leads

   Relevant Sections:
   - Project objectives and scope
   - Timeline and milestones
   - Resource allocation and budgetary considerations
   - Risk assessment and mitigation strategies
   - Quality assurance and testing plans

4. **Users**:
   - End-users (music enthusiasts and artists)
   - Moderators
   - Administrators

   Relevant Sections:
   - User interface specifications
   - Functional requirements related to user interactions (e.g., sharing songs, commenting)
   - Guidelines for moderators (e.g., managing comments, addressing issues)
   - Admin panel functionalities and permissions (e.g., user banning, content moderation tools)

By specifying the intended audience for each part of the SRS document, stakeholders, developers, project managers, and users can easily locate the information relevant to their roles and responsibilities, facilitating effective communication and collaboration throughout the project lifecycle.

1.5. Project Scope (Emmanuel)

The two main goals are to reduce the amount of time that a customer needs to wait and increase the amount of customers that the restaurant can serve in a day.

This aligns with the business aim of TuneTribe, which purports to be a social platform that enables music lovers to share their music and support emerging artists.

The benefits of this project to business include:

The environment is very important
Empowering the community to contribute: The TuneTribe platform enables users to share the best of their music, meet like-minded individuals, and discover new artists to ultimately attempt to maximize user engagement and loyalty.

Support upcoming artists: TuneTribe is for upcoming talents and seeks to support and promote them. This is by the fact that it allows the artists to have a home where they can upload their music and also develop a following. Driving user growth and retention: While interaction within the community is critical, exciting users with music discovery is poised to drive them to the platform and keep users on the platform longer over time.

1.6. Technology Challenges (Emmanuel)

Technology Challenges for a Web App:

1. Frontend performance: The frontend code and its rendering processes should be humanly optimized levels for fast load times with smooth user experience.

2. Browser Compatibility - All other browsers (e.g., Chrome, Firefox, Safari) and their versions must reach consensus and provide a single, united human experience across different platforms.

3. Responsive Design: Ensure that the web app is duly compatible with any screen size and device type seamlessly through implementing an appropriate responsive design.

4. Data Protection: Strong security measures are adopted for user data, such as encryption, secure communication protocols (HTTPS), and protections from common web vulnerabilities (e.g., XSS, CSRF).

5. Design scalable backend architecture that can adapt itself with the increasing number of users visiting and the data getting larger in volume without affecting the performance of the system or reliability of data.

6. Real-time communication of instant messaging or even an update feed with technologies such as WebSockets or Server-Sent Events.

7. Offline Functionality: Permit access to some of the features or content even if the user is not online.

8. Accessibility: Ensure the web app is accessible to all users, including persons with disabilities. The web app should be in line with accessibility guidelines (e.g., WCAG) and come with functionalities like keyboard navigability and screen reader friendliness.

9. Search Engine Optimization (SEO): This means making the web app optimized to get a high visibility and discoverability rating with required HTML markups, meta tags, and data structuring.

10. Continuous Deployment: You streamline the whole deployment process using a Continuous Integration/Continuous Deployment (CI/CD) pipeline that automates testing, deployment, and updating to guarantee lightning-fast and reliable releases.

1.7. References

Thelin, M. (n.d.). *Spotify-web-API-java/spotify-web-API-java: A java wrapper for Spotify's web API.* GitHub. https://github.com/spotify-web-api-java/spotify-web-api-java?tab=readme-ov-file

2. General Description

2.1. Product Features (Everyone)

The product features include the ability for users, artists, and moderators to create an account and the ability for administrators to manipulate those accounts. Users can create posts sharing the music they are listening to, recommend their favorite artists to their friends and discover new music through the artist discovery feature. Users will have their top artists tracked and displayed on their page. Artists are provided with dedicated spaces to upload their music, gain a following, and track royalties. Moderators can delete comments and posts that do not follow community guidelines, they can ban users for up to a week (temp ban). Moderators can also limit comments from a specific user who is not following community guidelines. If a user is a repeat offender, moderators can request to have the user permanently banned. An admin will be able to ban users, and artists, they will also have the ability to remove moderator and artists permissions.

2.2. User Class and Characteristics (Everyone)

User Class and Characteristics:

TuneTribe, as a website or web application, is designed to cater to users with diverse interests and roles within the music community. Users can be categorized into the following classes based on their characteristics:

1. **Music Enthusiasts**:
   - Music enthusiasts are individuals who have a passion for music and enjoy discovering new songs, artists, and genres.
   - They actively engage with the platform by sharing their favorite songs, creating music-related posts, and recommending songs/artists to others.
   - Music enthusiasts value features that facilitate music discovery, community interaction, and sharing their musical preferences with others.

2. **Casual Listeners**:
   - Casual listeners are users who enjoy listening to music casually without deep involvement in the music industry or community.
   - They may use the platform to explore new music, discover trending songs, and connect with friends through music-related interactions.
   - Casual listeners appreciate user-friendly features and intuitive interfaces that simplify music discovery and enhance their listening experience.

3. **Serious Music Fans**:
   - Serious music fans are individuals who have a deeper level of engagement with music, including a strong interest in specific genres, artists, or music scenes.
   - They actively participate in discussions, share insights, and engage with other users who share similar musical tastes and interests.
   - Serious music fans value platforms that provide opportunities for meaningful interactions, discovery of niche music, and connections with like-minded individuals.

4. **Emerging Artists**:

- Emerging artists are aspiring musicians who use the platform to showcase their music, gain exposure, and connect with fans.
- They utilize the dedicated space provided by the platform to upload their music, build a following, and track royalties.
- Emerging artists seek features that support their growth and development as musicians, including tools for promoting their music, engaging with fans, and tracking their success on the platform.

5. **Moderators and Administrators**:
- Moderators and administrators play a crucial role in ensuring the smooth operation of the platform and maintaining a positive user experience.
- They oversee user interactions, manage comments, address issues, and enforce community guidelines to foster a safe and welcoming environment for all users.
- Moderators and administrators require access to specialized tools and privileges to effectively perform their duties and maintain platform integrity.

By catering to the needs and preferences of these user classes, TuneTribe aims to create a vibrant and inclusive community where music enthusiasts and artists can connect, discover, and share their passion for music with others.

Our website application does not expect our users to have any prior knowledge of a computer, apart from using a web browser, or knowledge of astronomy. Our website application has removed the need for them to have astronomy, math, or science knowledge and allows the user to focus on exploring the night sky.

2.3. Operating Environment (Everyone)

The application is designed to operate on the web across many different devices.

2.4. Constraints (Everyone)

Due to TuneTribe being a webapp, fluent mobile access may be limited. Interaction with website is already limited on mobile.

2.5. Assumptions and Dependencies (Everyone)

The software will be dependent on Spring Web in order to create and execute the MVC architecture that will be developed within NetBeans, VS Code, and Spring. These technologies will facilitate the development of a robust and scalable web application, providing essential features for handling client requests, managing server-side logic, and rendering dynamic HTML content. NetBeans and VS Code will serve as the primary integrated development environments (IDEs) for coding and debugging, offering extensive tooling support and seamless integration with the Spring framework. Additionally, Spring's comprehensive ecosystem of libraries and tools will enable efficient development, testing, and deployment of the software, ensuring high performance and reliability throughout the development lifecycle.

The software will be dependent on Spring Web and Thymeleaf in order to create and execute the MVC architecture that will be developed within NetBeans, Vs Code, Spring, .

3. Functional Requirements

3.1. Primary (Shauna)

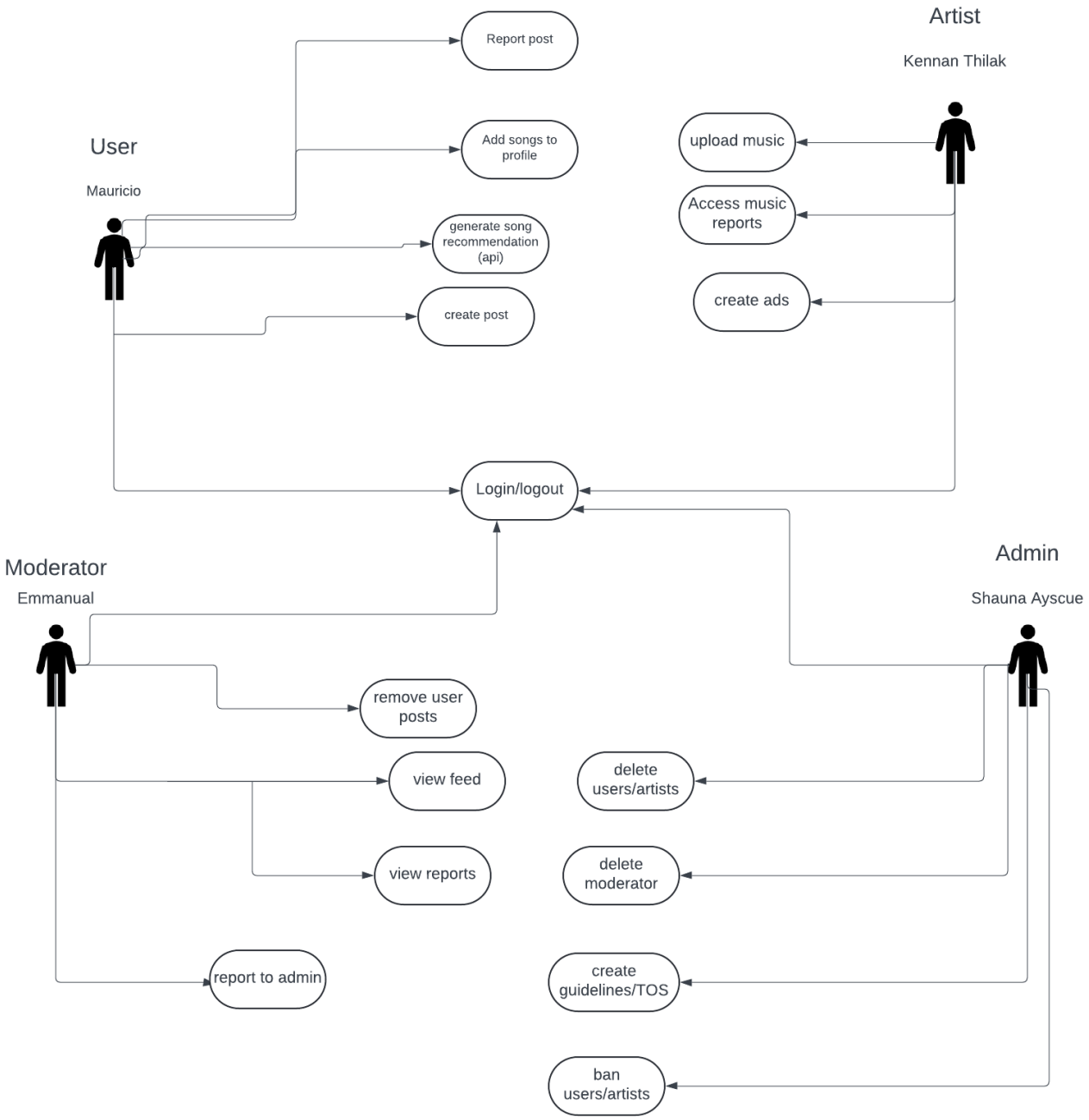
-The System will allow the user to search for songs based on Song or artist name
-This System will allow user to add songs to profile.
-This System will users to recommend music
-This System will users to create posts
-This System will users to register
-The System will allow users to create and login to their accounts using their personal information and log out
-The System will allow users with artist partnership to input their song information which will boost their post and their music within the app
-This System will allow admin to bans/delete users and artist, delete moderators, create TOs and Community Guidelines.
-This System will moderators to view reports made from users and view the feed that has been reported.
-This System will artists to upload music, create advertisements, and access music reports.

3.2. Secondary (Shauna)

- Password protection for users, artists, moderator, admin.
- System to keep track of users/artists who have been banned or limited from logging in.

3.3 Use-Case Model

3.3.1 Use-Case Model Diagram (Shauna)



3.3.2 Use-Case Model Descriptions

3.3.2.1 Actor: User (Mauricio)

- **Generate Artist Recommendation:**
  - **User can request an song recommendation based on the top song listen on their profile**
- **Create Post:**
  - **Users can create a post**
    - **Post will consist of**
      - **Caption**
      - **username**
      - a song
- **Recommend Artists/Music:**
  - **Users can recommend a song/artist .**

-
- **Add Songs to Profile:**
    - **Users can add songs they like to their profile for others to see.**
- **Report Users:**
    -Users can report other users to moderator.
- **Login:**
    - **User will be able to log into their accounts and sync across other browsers**
- **Register:**
    -Users can register with their email, password, username, first/last name

3.3.2.2 Actor: Admin (Shauna)

- **Ban Users**: The admin can prohibit users from accessing TuneTribe due to violations of community guidelines or inappropriate behavior.
- **Ban Artists**: Artists who breach terms of service or upload unauthorized content can be banned from TuneTribe by the admin, resulting in the removal of their content and account restrictions.
- **Delete Moderator**: The admin has the authority to delete moderator ultimately removing their account.
- **Create Community Guidelines**: The admin can create community guidelines for Tune Tribe to ensure a standard of behavior for its users.
- **Create Terms of Service:** The admin can create terms of service for Tune Tribe that will show a set of rules that the users must agree to when using the platform.
- **Login**: Admins can access the admin dashboard by logging in with valid credentials, enabling them to perform administrative tasks and manage the TuneTribe platform.

3.3.2.3 Actor: Artist (Kannan)

- Access Music Reports: Artists can access detailed reports about the performance of their music, including streaming statistics, demographic insights, and geographical reach.
- Upload Music to App: Artists can upload their music to the application, making it available for streaming and discovery by users.
- Create advertisements: Artists can create advertisements to boost their music.
- Login: Artists can log into their accounts to manage their profile, upload new music, access reports, and engage with their audience. Their login credentials will allow them to sync their account across different browsers for seamless access.

3.3.2.4 Actor: Moderator (Emmanuel)

- **View Reports of Users**: Moderators enable users to report instances of rule violations or misconduct by other users, facilitating community-driven enforcement of platform policies.
- **View Feeds:** Moderators can view the feed of users to see what they have posted
- **View Feeds:** Moderators can view the feed of users to see what they have posted
- **Login**: Moderators can access their moderator dashboard and perform moderation tasks upon successful authentication through the login process.

3.3.3 Use-Case Model Scenarios

3.3.3.3 Actor: User (Mauricio)

- **Use-Case Name**: Generate Artists Recommendation
  - **Initial Assumption**: User has access to artists recommendation and can have a song recommended to them
  - **Normal**: The user is able to generate an artists recommendation based on the songs on the user's profile
  - **What Can Go Wrong:** User does not have any artists/songs added to their profile
  - **Other Activities**: User may be asked to add songs to their profile
  - **System State on Completion**: A new page is loaded along with a new artist for the user to listen to.
- **Use-Case Name**: Create post
  - **Initial Assumption**: User has access create post feature
  - **Normal**: User will create a post which will consist of an emoji and a song
  - **What Can Go Wrong:** The song the user is looking for is not found
  - **Other Activities**: The user can check for spelling errors
  - **System State on Completion**: A new post is added to the user's timeline
- **Use-Case Name**: Login
  - **Initial Assumption**: The user has access to login feature
  - **Normal**: The user logs in an they have access to their account
  - **What Can Go Wrong:** The users credentials cannot be validated
  - **Other Activities**: User is redirected to enter credentials again or to create an account
  - **System State on Completion**: The user will have access to the web app
- **Use-Case Name**: Add Song to profile
  - **Initial Assumption**: User has access to this feature
  - **Normal**: The user will be able to add up to 5 songs to their profile for display purposes
  - **What Can Go Wrong:** The song the user is looking for is not found
  - **Other Activities**: prompt user to search again
  - **System State on Completion**: The requested song will appear on the users profile
- **Use-Case Name**: Make Reports
  - **Initial Assumption**: User has access to this feature
  - **Normal**: The user will be able to report others users.
  - **What Can Go Wrong:** The report is not sent.
  - **Other Activities**: prompt user to report again
  - **System State on Completion**: The report will appear on the moderators dashboard.
-

3.3.3.4 Actor: Moderator (Emmanuel)

- **Use Case Name: View Reports**
  - **Initial Assumption**: The moderator is logged into the moderator dashboard and has the authority see reported users.

- **Normal**: The moderator identifies a user who has violated community guidelines and deletes the user.
- **What Can Go Wrong**: Accidentally deleting the wrong user due to a selection error. Technical issues prevent the moderator from accessing the moderation dashboard.
- **Other Activities**: Reviewing the user's activity history before deciding to delete.
- **System State on Completion**: The deleted user is restricted from accessing the platform.

- Use Case Name: **View User Feed**
  - **Initial Assumption**: The moderator is logged into the moderator dashboard and can view the feed of users.
  - **Normal**: The moderator identifies a user who has been reported.
  - **What Can Go Wrong**: Mistakenly viewing the feed of a user who has not violated guidelines. Technical issues prevent the moderator from accessing the moderation dashboard.
  - **Other Activities**: Communicating with the user regarding the reason for being reported.
  - **System State on Completion**: The user's who are reported are warned about their report.
- Use Case Name: **Remove User posts**
  - **Initial Assumption**: The moderator is logged into the moderator dashboard and can be able to remove a user post.
  - **Normal**: The moderator identifies a user who has been removed.
  - **What Can Go Wrong**: Mistakenly removing the user who has not violated guidelines. Technical issues prevent the moderator from accessing the moderation dashboard.
  - **Other Activities**: Communicating with the user regarding the reason for being removed.
  - **System State on Completion**: The user's who are removed is deleted and cannot log into account.
- Use Case Name: **Report to Admin**
  - **Initial Assumption**: The moderator is logged into the moderator dashboard and can report to admin if they think it's necessary.
  - **Normal**: The moderator sends a report to the admin.
  - **What Can Go Wrong**: Mistakenly sending the wrong report to the admin who has not violated guidelines.
  - **Other Activities**: Communicating with the admin regarding the reason for the report.
  - **System State on Completion**: The admin receives the report from the moderator.

3.3.3.5 Actor: Artist (Kannan)

- **Use-Case Name**: Upload Music
  - **Initial Assumption**: The artists has access to the Tune Tribe platform and is logged in as an artist.

- **Normal**: The artists navigates to the upload music section and selects the music file from his device. He fills in the required details such as song title, artist name, genre, and album information. Kennan then confirms the upload, and the system processes the music file.
- **What Can Go Wrong**: If the artists attempts to upload a music file in an unsupported format. The system should display an error message indicating the supported file formats and prompt Kennan to upload a compatible file.
- **Other Activities**: The artists can edit the details of the uploaded music later, including updating the title, genre, or album information.
- **System State on Completion**: The uploaded music is successfully added to Artist's profile on the platform, where it is available for other users to discover and listen to.

- **Use-Case Name**: Access Music Reports
  - **Initial Assumption**: The artist has access to the Tune Tribe platform and is logged in.
  - **Normal**: the artist navigates to the music reports section to view detailed reports on the performance of their uploaded music. They can analyze metrics such as streams, downloads, likes, and comments for each track. The reports may also include insights into audience demographics and engagement patterns.
  - **What Can Go Wrong**: The artist encounters discrepancies or inaccuracies in the reported music performance metrics. The system should provide a mechanism for the artist to report the issue and request further investigation by platform administrators. Additionally, technical issues such as data synchronization errors or server downtime may affect the availability or accuracy of the reports.
  - **Other Activities**: The artist can download the music reports for their records or export them in various formats for further analysis. They may also share the reports with collaborators or stakeholders to demonstrate the success and impact of their music on the platform.
  - **System State on Completion**: The artist has access to comprehensive and reliable information on the performance of their music uploads, enabling them to make informed decisions, track their success, and optimize their strategies for maximizing engagement and revenue on the platform.
- **Use-Case Name**: Create Advertisements
  - **Initial Assumption**: The artist has access to the Tune Tribe platform and is logged in.
  - **Normal**: the artist creates advertisements to boost their music on the app. They can post their ads on the home pages and its can be seen by other artists and users.
  - **What Can Go Wrong**: The artist encounters inaccuracies in the ads they posted. Additionally, technical issues such as data synchronization errors or server downtime may affect the availability or accuracy of the ads.
  - **Other Activities**: The artist can see how many users have viewed the ad.

- **System State on Completion**: The artist has access to comprehensive and reliable based on the ads that they created and can track their success.

3.3.3.6 Actor: Admin (Shauna)

- **Ban Users**:
  - **Initial Assumption**: The admin has access to ban a user from TuneTribe if policies are violated.
  - **Normal**: The admin will select a certain user to ban and prevent them from commenting or posting on the webapp.
  - **What Can Go Wrong**: The admin could accidently ban the wrong user.
  - **Other Activities**: The admin could ban users from commenting on users post.
  - **System State on Completion**: The banned user is deleted from the platform with the admin logs being updated.
- **Ban Artists**:
  - **Initial Assumption**: The admin has access to ban an artist from TuneTribe if policies are violated.
  - **Normal**: The admin determines what artist has violate the guidelines of the webapp and selects the artist to be banned.
  - **What Can Go Wrong**: The admin could accidently permanently ban the wrong artists due to an error in the system.
  - **Other Activities**: Reviewing the artist that has been banned by searching their history.
  - **System State on Completion**: The selected banned artists are restricted from accessing the webapp permanently.
- **Delete Moderator**:
  - **Initial Assumption**: The admin has the authority to delete the moderator from Tune Tribe.
  - **Normal**: The admin cannot allow a moderator to access their account anymore.
  - **What Can Go Wrong**: The admin could choose the wrong moderator to delete.
  - **Other Activities**: The admin could re-add a moderator.
  - **System State on Completion**: The moderators' tasks are deleted, and the admin's log is up to date.
- **Delete Artists**:
  - **Initial Assumption**: The admin has the authority to delete an artist at any time if they violate any policies.
  - **Normal**: The admin can delete an artist and they won't be able to create an account anymore.
  - **What Can Go Wrong**: The admin could delete the wrong artist due to misunderstanding of guidelines.
  - **Other Activities**: The admin could contact the artist and determine the reasons for deleting the artist.
  - **System State on Completion**: The artists tasks are deleted, and the admin's log is up to date.
- **Delete Users**:

- **Initial Assumption**: The admin has the authority to delete a user at any time if they violate any policies.
- **Normal**: The admin can delete a user and they won't be able to create an account anymore.
- **What Can Go Wrong**: The admin could delete the wrong user due to misunderstanding of guidelines.
- **Other Activities**: The admin could contact the user and determine the reasons for deleting the artist.
- **System State on Completion**: The user tasks are deleted, and the admin's log is up to date.

- **Create Community Guidelines**:
  - **Initial Assumption**: The admin can create guidelines that users and artists need to follow if they wish to keep their account.
  - **Normal**: The admin can create guidelines that can be viewed by users and artists
  - **What Can Go Wrong**: The admin could post the wrong community guidelines.
  - **Other Activities**: The admin could update the community guidelines if they need to.
  - **System State on Completion**: The updated guidelines are posted for users and artists.
- **Create Terms of Service**:
  - **Initial Assumption**: The admin can create terms of service that users and artists need to follow if they wish to keep their account.
  - **Normal**: The admin can create terms of service that can be viewed by users and artists
  - **What Can Go Wrong**: The admin could post the wrong terms of service.
  - **Other Activities**: The admin could update the terms of service if they need to.
  - **System State on Completion**: The updated terms of service are posted for users and artists.

4 Technical Requirements

4.3 Interface Requirements (Shauna)

4.3.2 User Interfaces (Shauna)

Users will have access to:

- Nav bar
  - Profile
  - Search
  - Create posts
  - Community guidelines
  - Terms of service
- Search
  - Here they can search
    - Other users
    - Songs
- Create post
  - Users will be prompted to include
    - caption
    - A song

Admins will have access to:

- Nav bar
  - Tune Tribe
  - Community guidelines
  - Terms of service
- User List
  - Here they can view:
    - Ban users
    - Delete users
- Artist List
  - Here they can view:
    - Ban artists
    - Delete artists
    - See id, username, first name, last name, email, status, remove
- Mod List
  - Here they can view:
    - Delete mods
    - See id, username, first name, last name, email, remove
- Community Guidelines
  - Here they can view:
    - Text table for editing community guidelines
    - Edit/save button
    - Bold, italic,underline,paragraph,large text,medium text,small text for styling
- Terms of Service
  - Here they can view:
    - Text table for editing terms of service

        - Edit/save button
        - Bold, italic,underline,paragraph,large text,medium text,small text for styling

Moderators will have access to:

- Here they can view moderator dashboard:
    - Incoming reports
    - Current post feed
    - Push user ban
    - Home button

Artists will have access to:

- [Missing Artist Input]

4.3.3 Hardware Interfaces (Shauna)

Tune Tribe will run on laptops/desktops that has access to the internet, the ability to display webpages, and the ability to interact with web pages.

4.3.4 Communications Interfaces (Shauna)

It must be able to connect to the internet as well as the local database on phpMyAdmin. The communication protocol, HTTP, must be able to connect to the Spotify API and return the current date and time.

4.3.5 Software Interfaces (Shauna)

We will use Spring Boot, ThymeLeaf to help build the frontend, as well as JPA for the backend database functionality. We will also use Spring Boot with Java to connect the frontend to the backend. We will also be using a MySQL DataBase to store all data.

5 Non-Functional Requirements

5.3 Performance Requirements (Emmanuel)

-NFR0(R): The TuneTribe platform should maintain optimal memory usage, ensuring that the local copy of the music database consumes less than 50 MB of memory.

-NFR1(R): The system, including all components and databases, should operate efficiently within memory constraints, consuming less than 100 MB of memory under normal usage conditions.

-NFR2(R): Novice users should be able to browse, search for songs, and create playlists within the TuneTribe platform in less than 10 seconds.

-NFR3(R): Expert users, familiar with the platform's interface, should be able to perform advanced tasks such as uploading music, managing their artist profile, and engaging with the community in less than 30 seconds.

5.4 Safety Requirements (Emmanuel)

- Safeguards should be implemented to protect user data and privacy, including encryption of sensitive information and secure authentication mechanisms.
- Measures must be in place to prevent unauthorized access to user accounts and ensure the integrity of user-generated content.
- Banning Users to uphold the community's enviornment and integrity.

5.5 Security Requirements (Emmanuel)

- NFR4(R): Access to the TuneTribe platform should be restricted to authorized users and their specified roles only, requiring valid authentication credentials for login and access to system functionalities.

5.6 Software Quality Attributes (Emmanuel)

5.6.2 Availability

The platform should strive for high availability, minimizing downtime and ensuring uninterrupted access for users.

5.6.3 Correctness

All functionalities and data processing should be accurate and error-free to maintain the integrity of the platform.

5.6.4 Maintainability

The codebase should be well-structured and documented, facilitating easy maintenance and updates by developers.

5.6.5 Reusability

Common components and functionalities should be designed for reusability across different modules to promote efficient development and reduce redundancy.

5.6.6 Portability

The platform should be compatible with various devices, browsers, and operating systems, ensuring a consistent user experience across different platforms.

5.7 Process Requirements (Mauricio)

5.7.2 Development Process Used

Scrum

5.7.3 Time Constraints

- We are given two months to complete the project

5.7.4 Cost and Delivery Date

- Cost will be 0
- Delivery date by the end of the semester

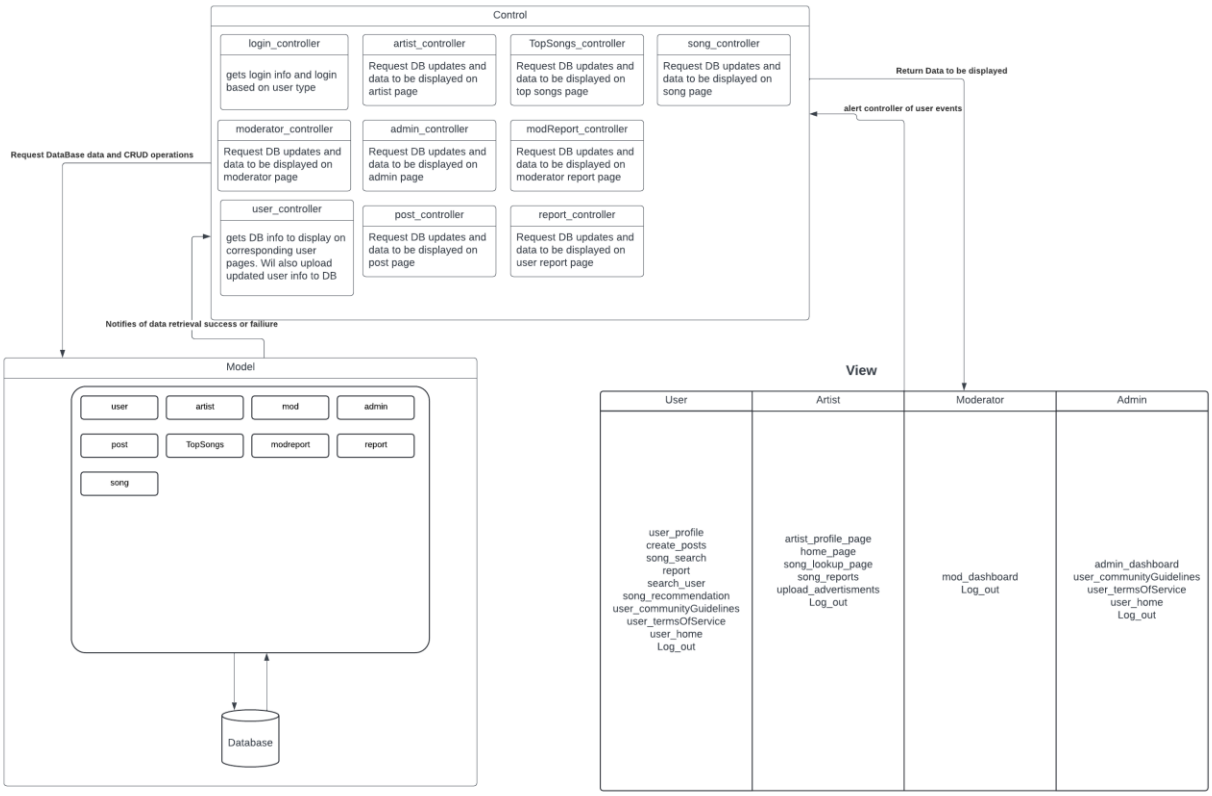
5.8 Other Requirements (Emmanuel)

-There should be frequent communication among team members about the project's status, requirements, and goals to keep the progress of work cohesive.

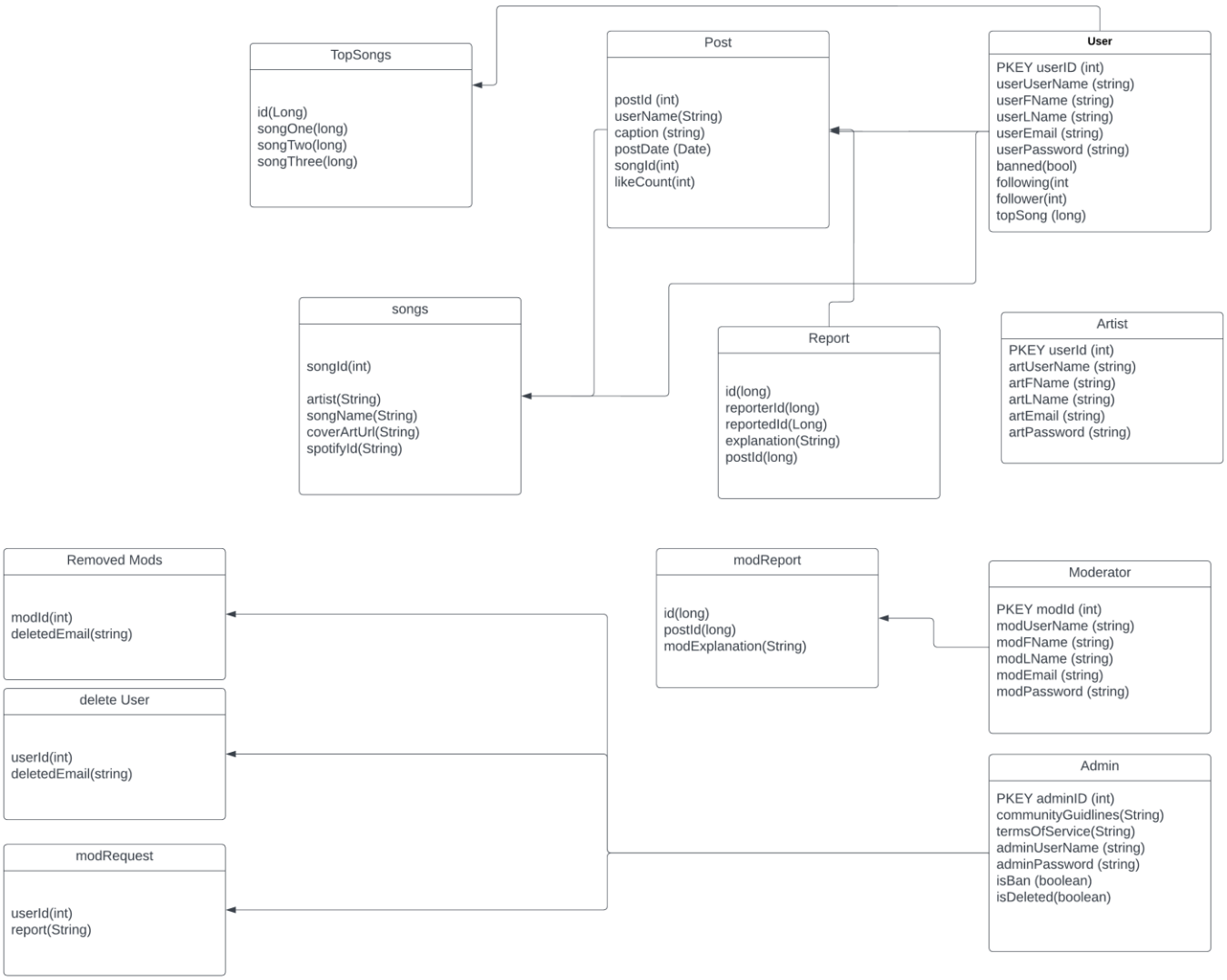
-Documentation of codebase and project progress shall be done to help in knowledge transfer amongst team members, and this shall guide them for future platform maintenance.

6 Design Documents

6.3 Software Architecture (Everyone)

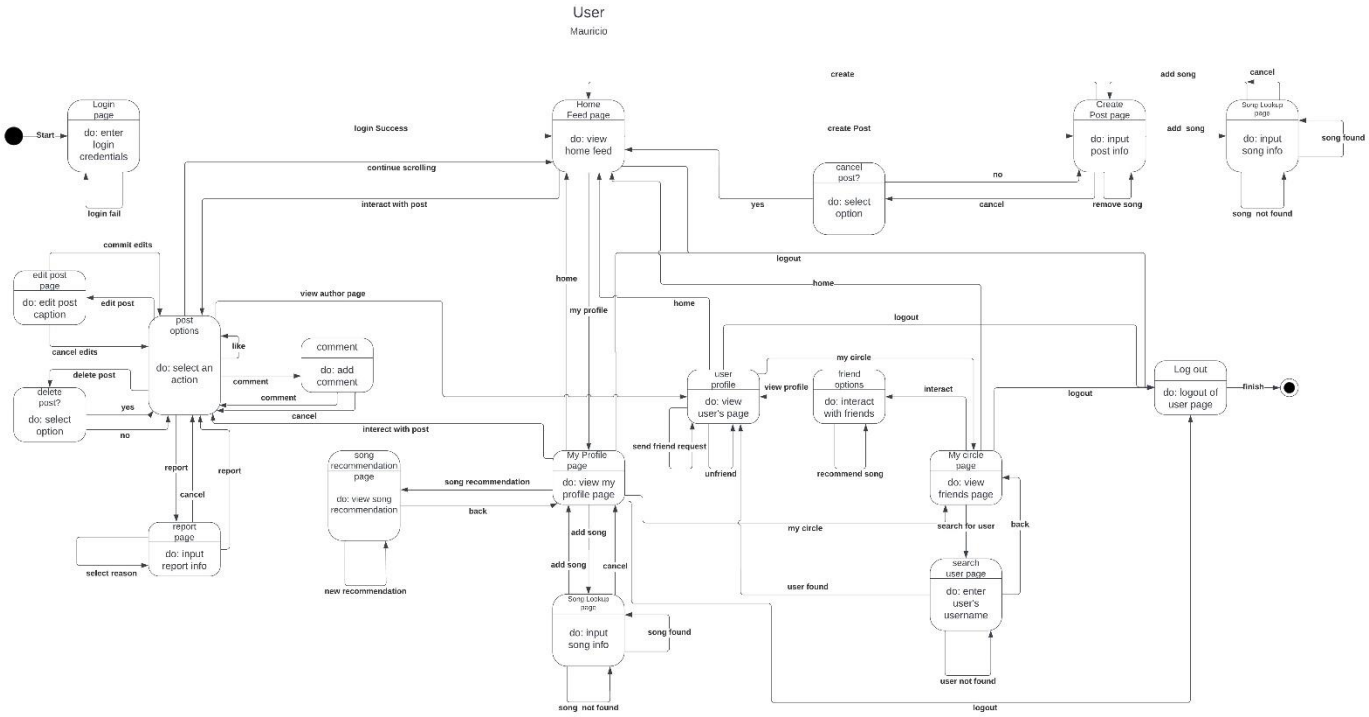


6.4 High-Level Database Schema (Everyone)

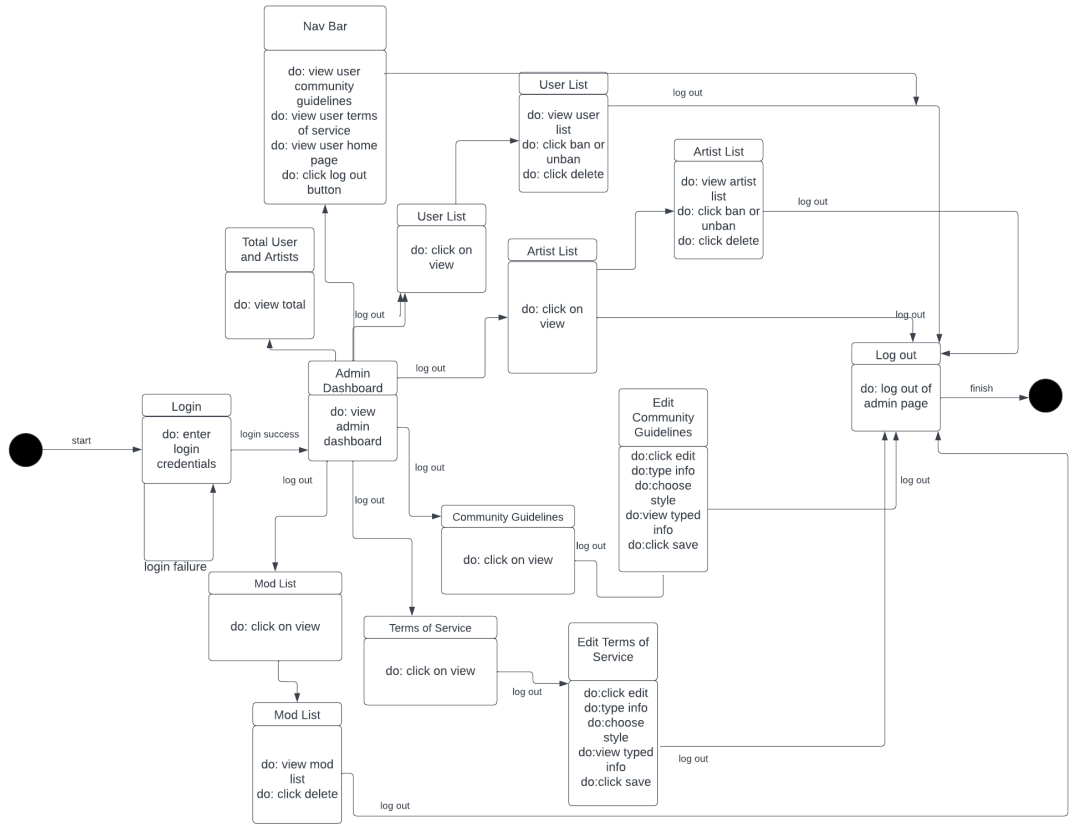


6.5 Software Design

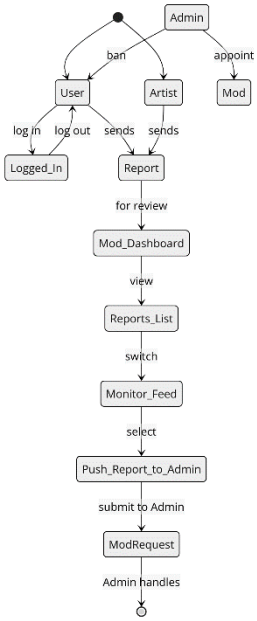
6.5.2 State Machine Diagram: User (Mauricio)



6.5.3 State Machine Diagram: Admin (Shauna)



6.5.4 State Machine Diagram: Emmanuel (Mod)

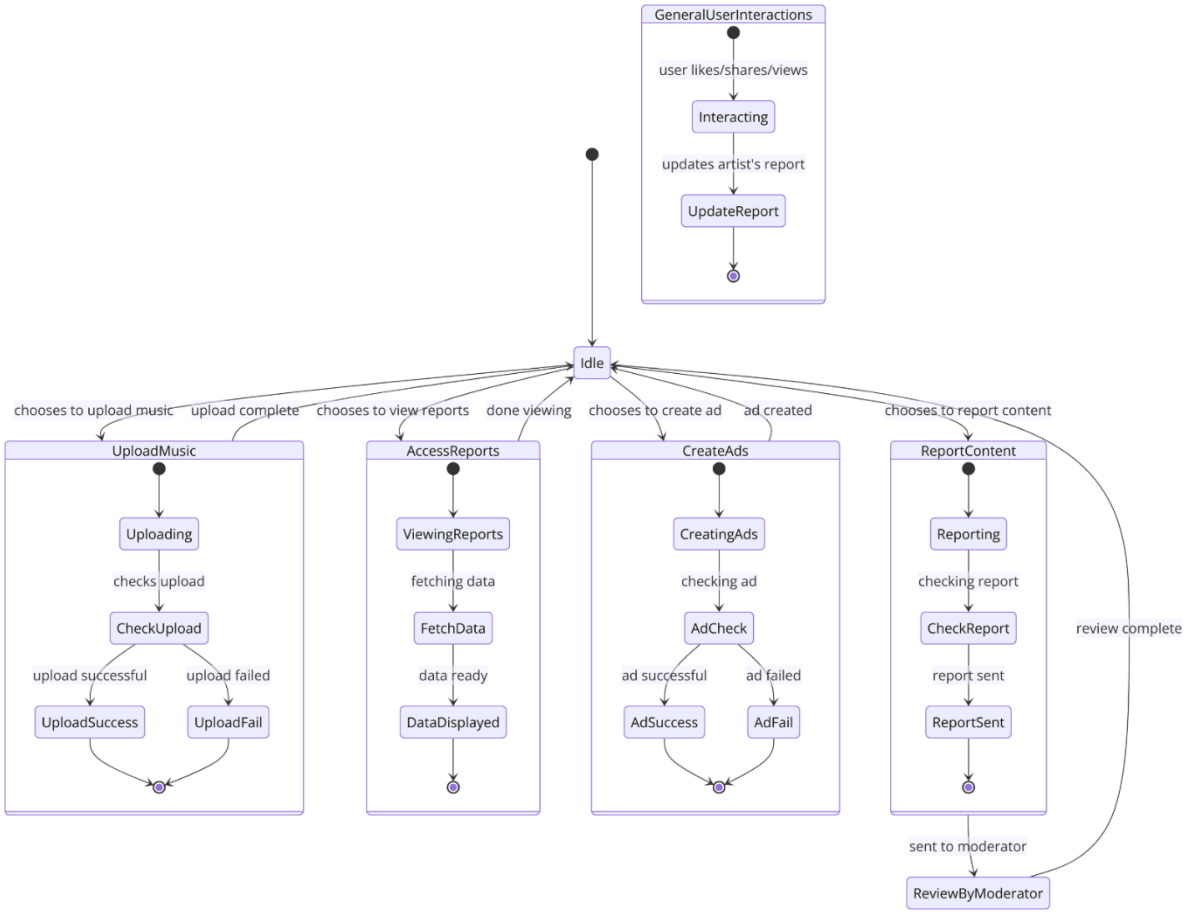


.

.

6.5.5 State Machine Diagram: Artist (Kennan)

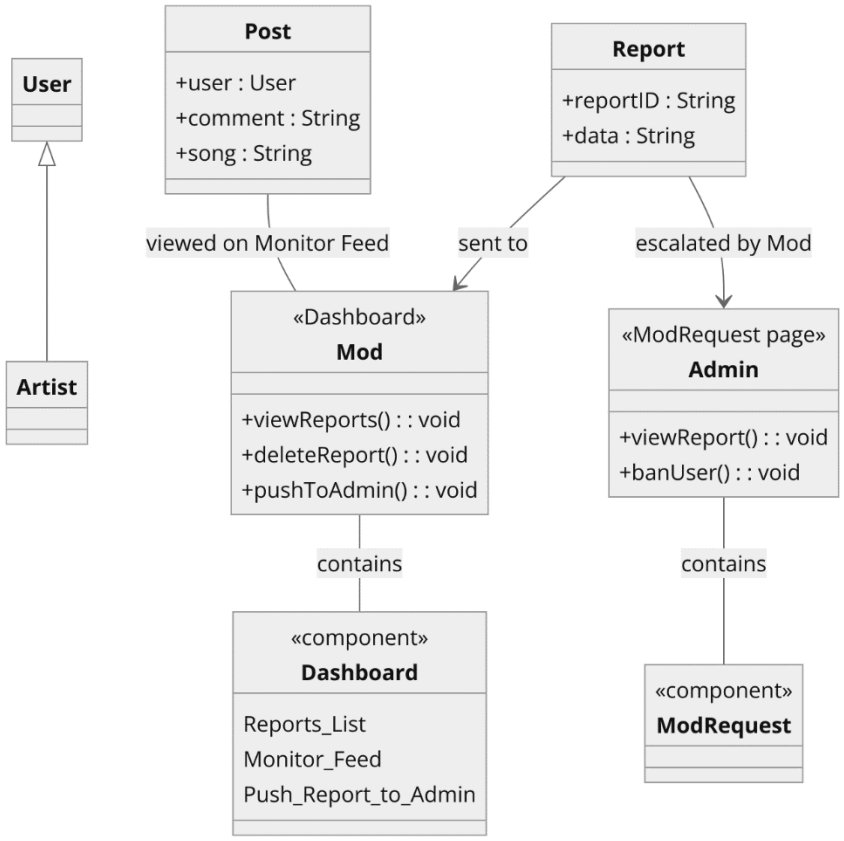
.



.

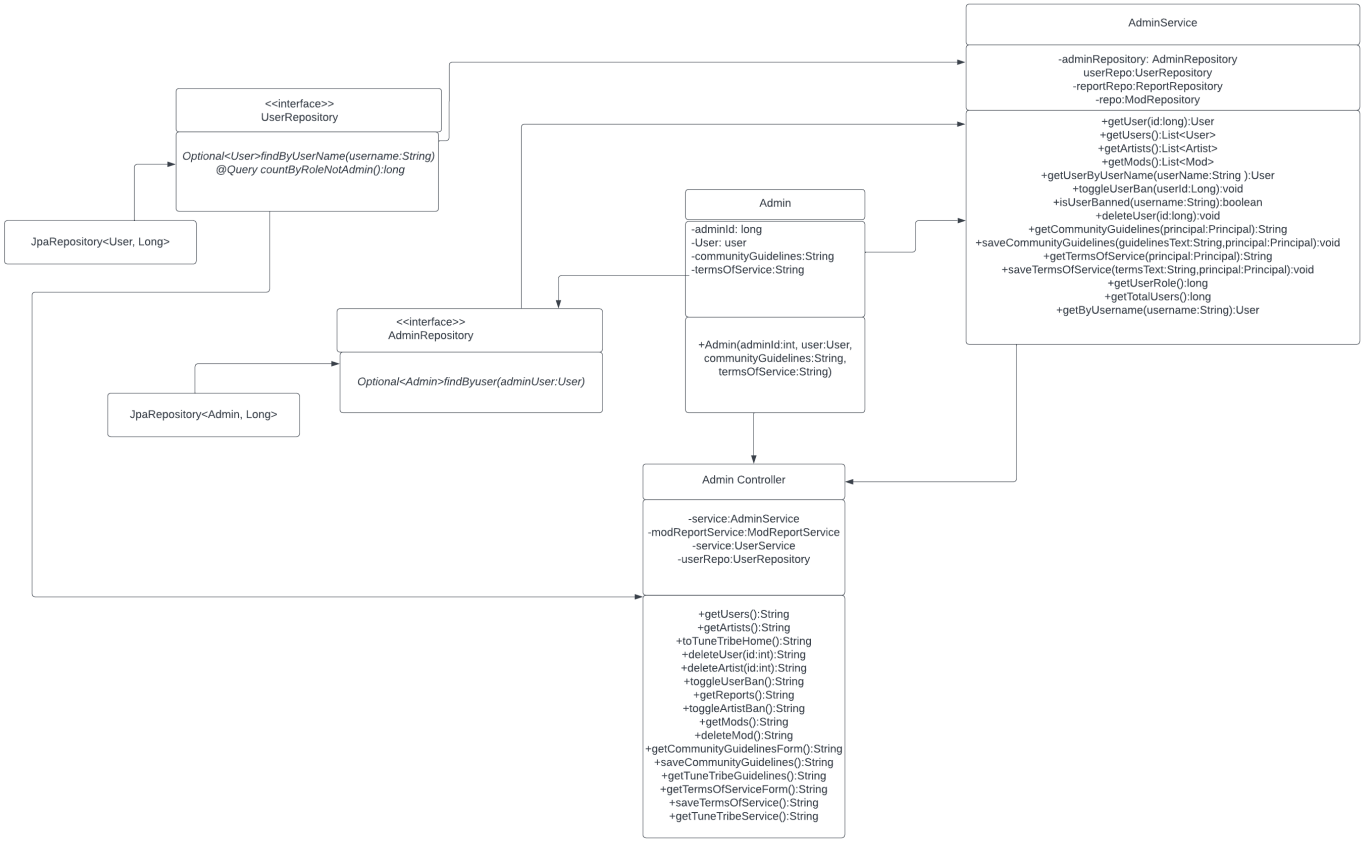
.

6.6 UML Class Diagram
Mod – Emmanuel



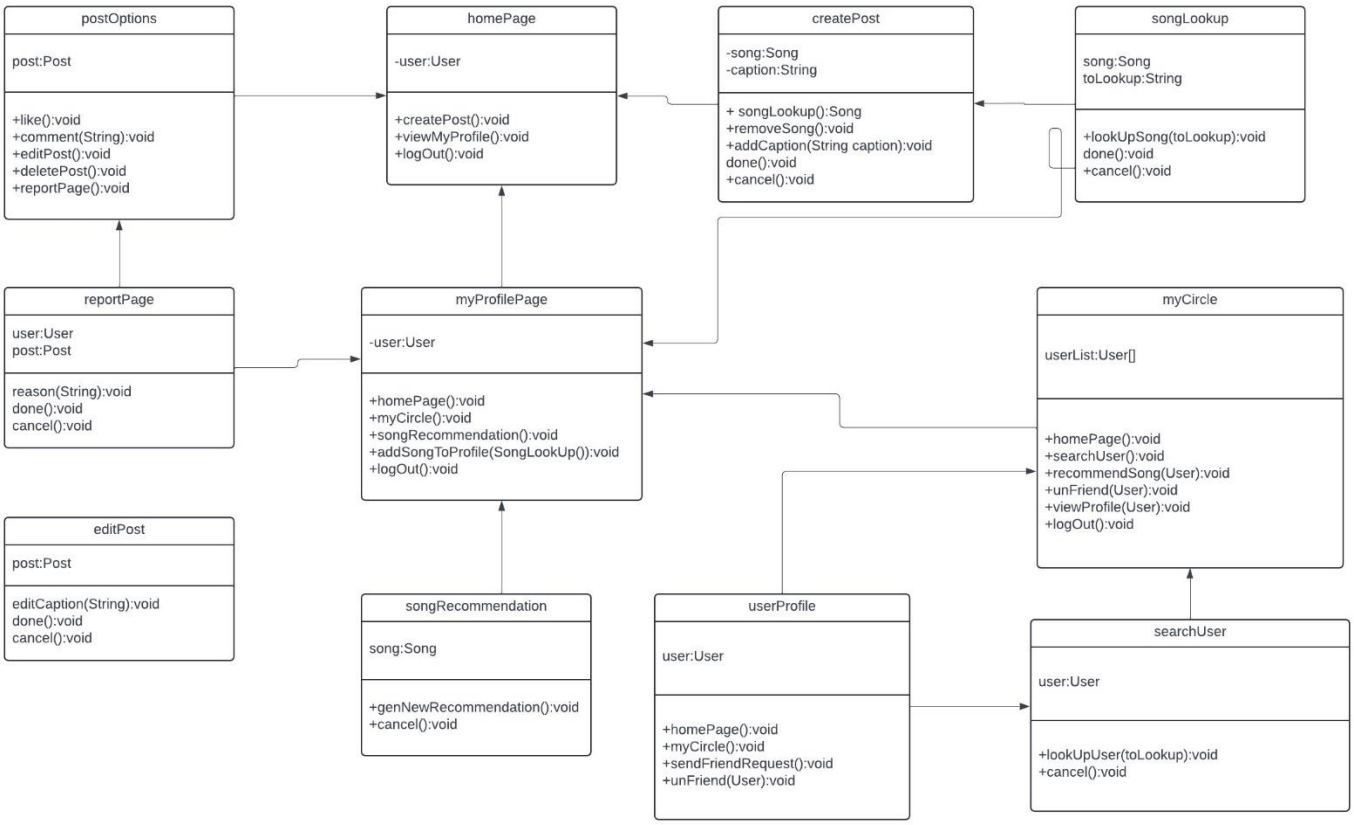
.

Admin- Shauna Ayscue



Artist-Kennan
(Missing)

User-Mauricio

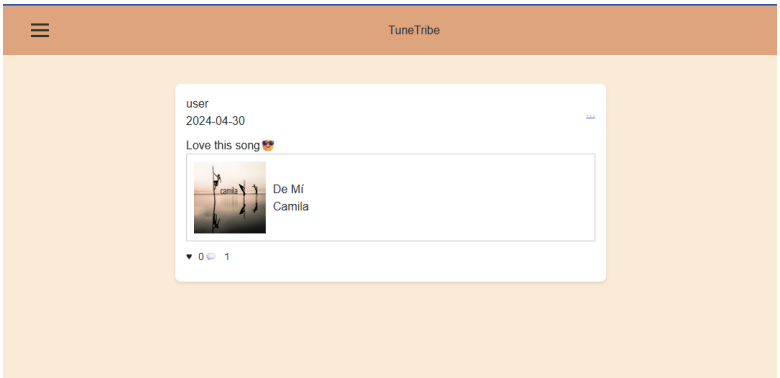


ss

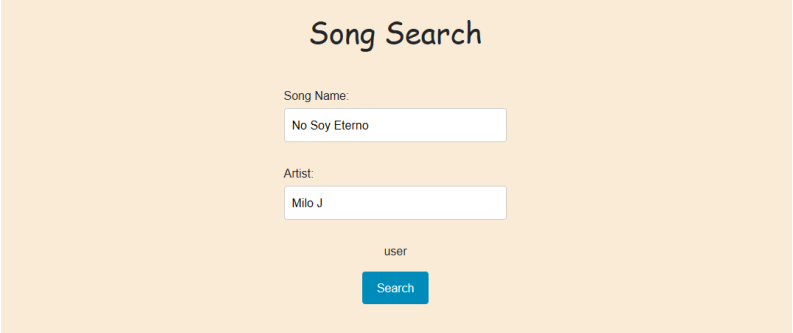
7 Scenario

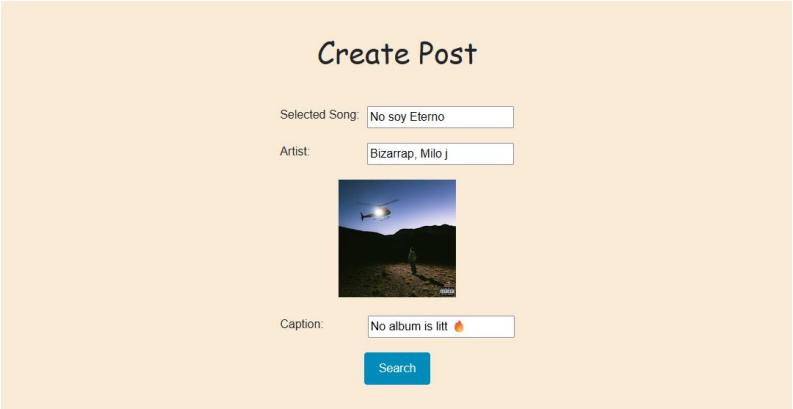
7.3 Brief Written Scenario with Screenshots

A user will log in, and see their home page with one post.

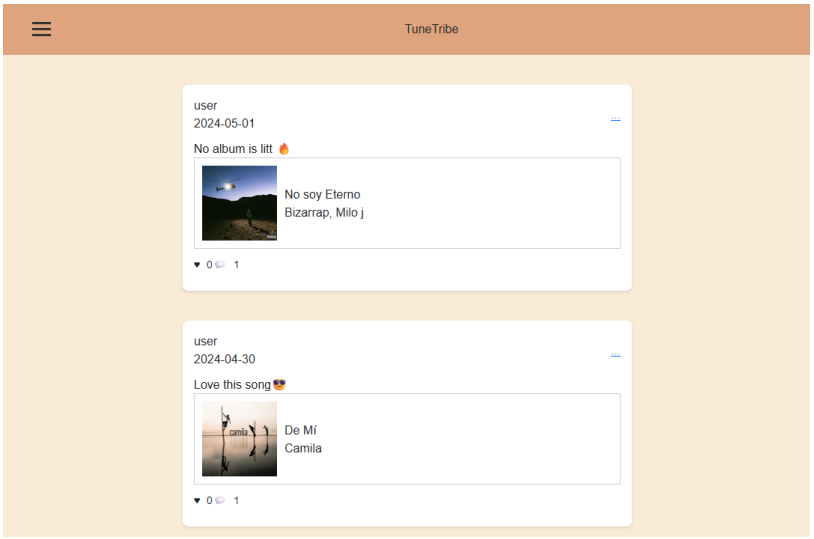


User will then create a post by searching for a song and adding a caption after a song is found

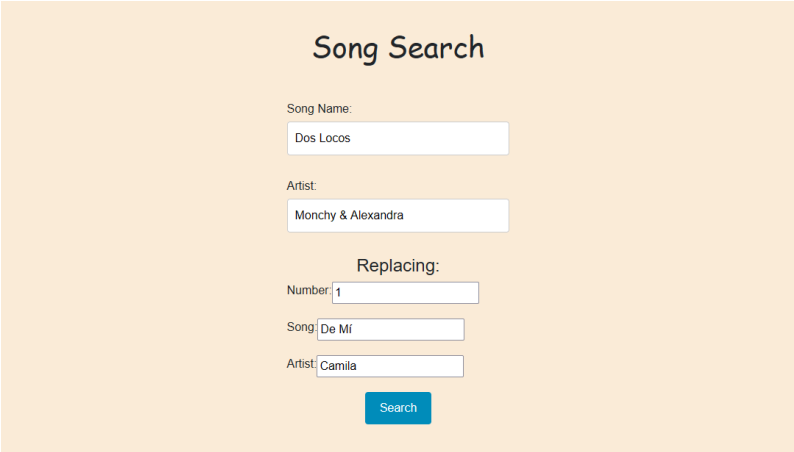




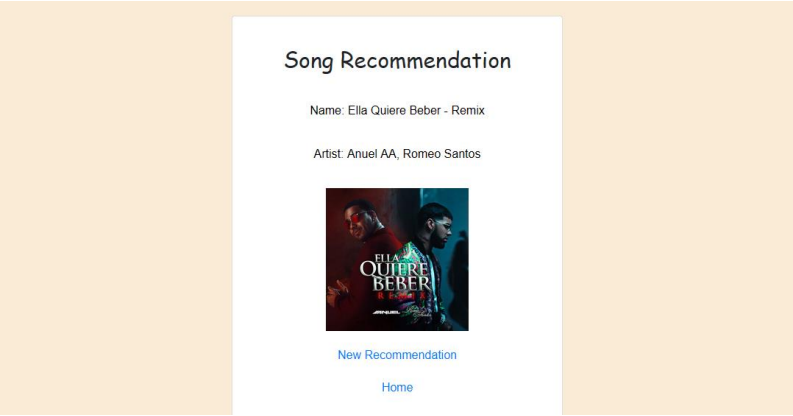
After, the new post should appear in the home feed. And I will show the user profile page where they user will have the default songs as their top songs.



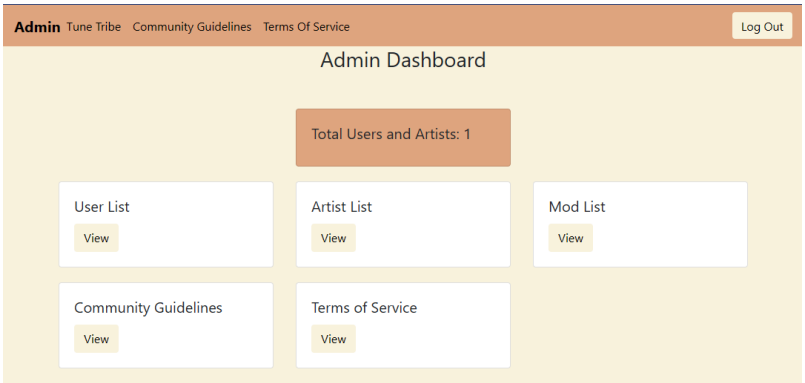
I will now show how to update your top songs by clicking album art of the song I want to change. This will prompt me to search for a song which will replace the previous song. It should also display the song you are replacing but that is not working. The song should appear on my profile page

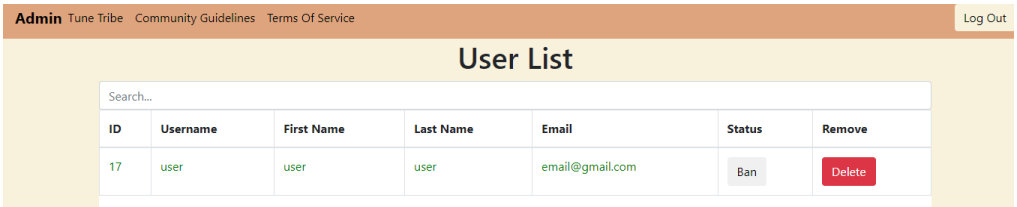


I will then demonstrate the song recommendation which will use the top song as a seed. This is done using the “Surprise me” button

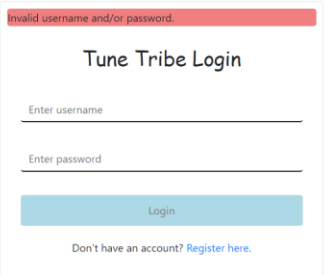


Next is admin. Once Admin logs in they can see their Dashboard which will show the total number of users and artists at top. Admin will then go to the user list and demonstrate banning a user

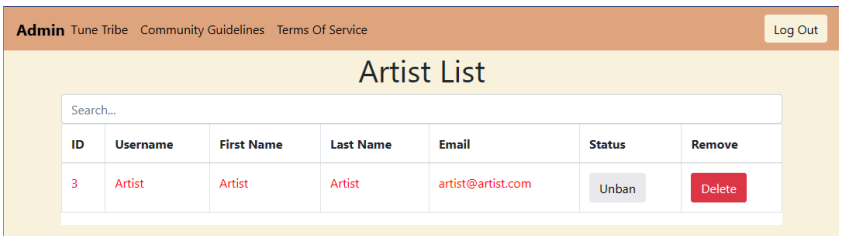


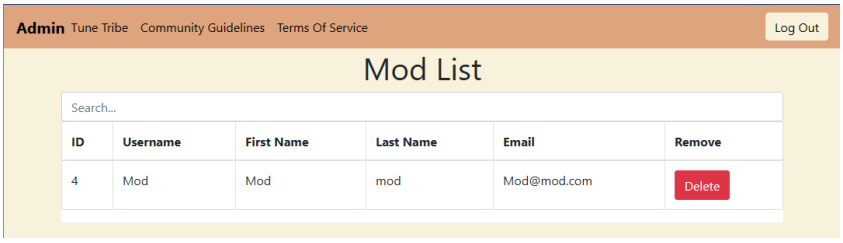


After getting banned, user will try to log in but they wont be able to. Admin then unbans users.

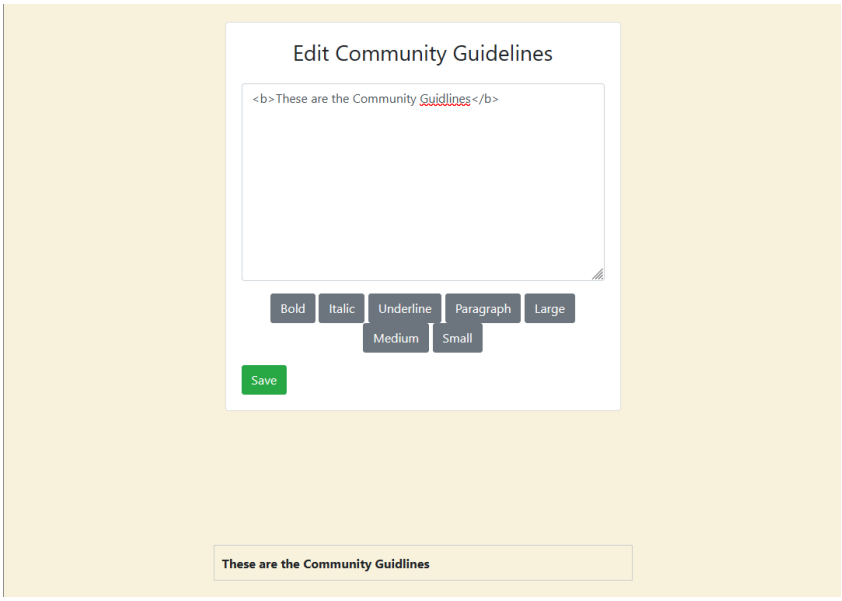


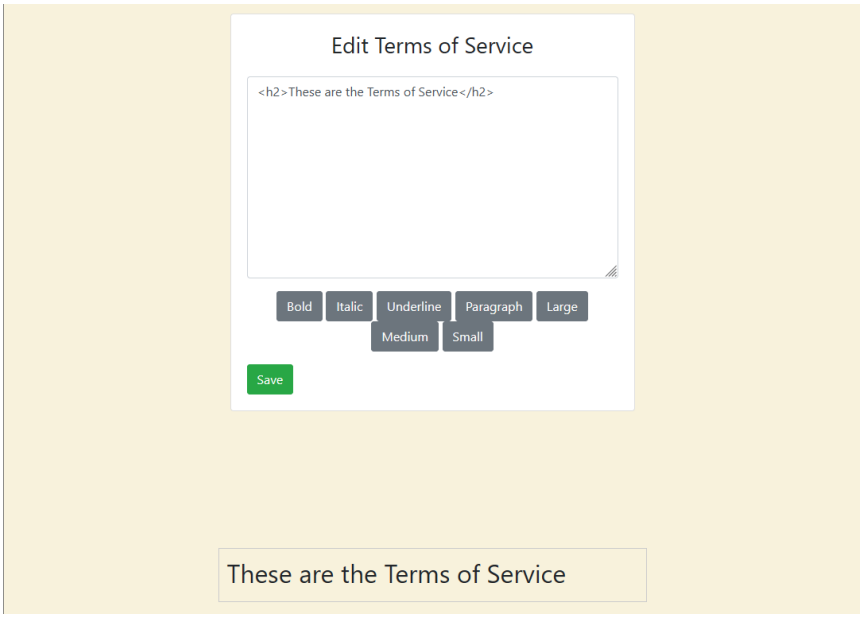
Admin then shows the Artist list where Admin can also ban or delete an artist. And the mod list where Admin can delete a mod.



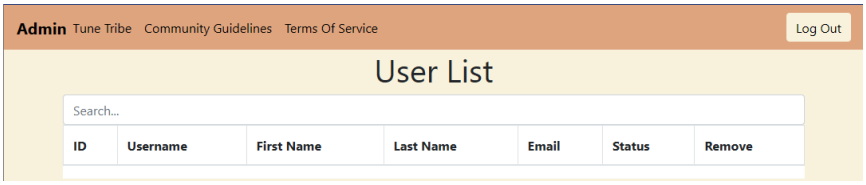


Next, the Admin will show how to create/ add to the community guidelines and the terms of service.

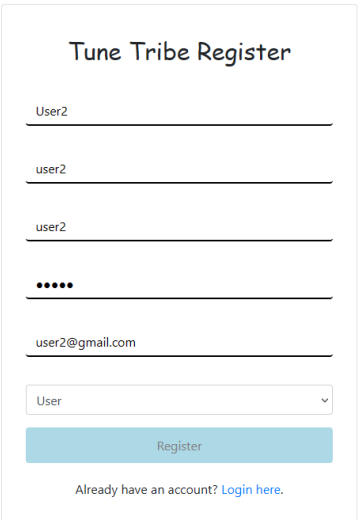




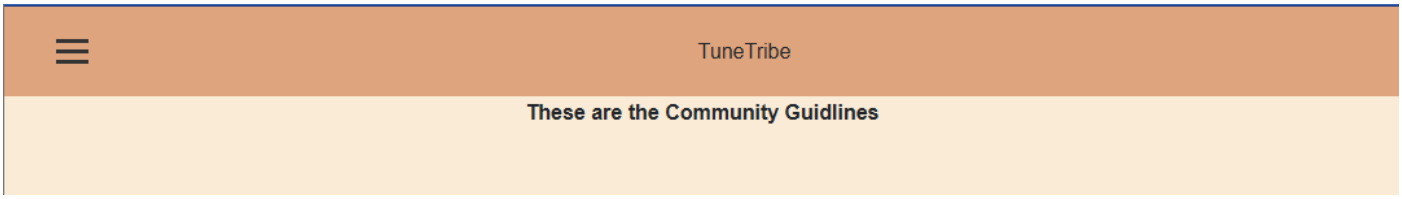
Finally, Admin deletes a User.

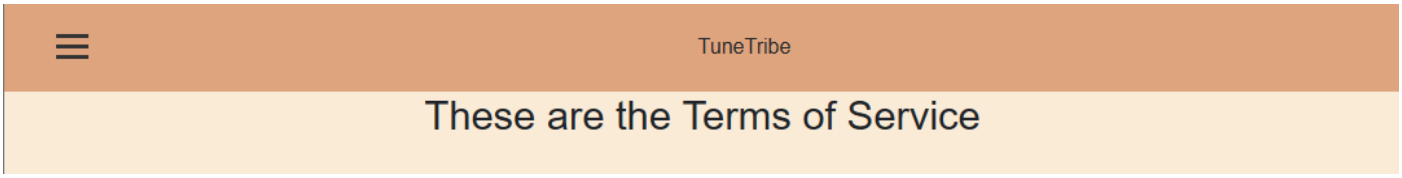


After that, we show a new user being created.



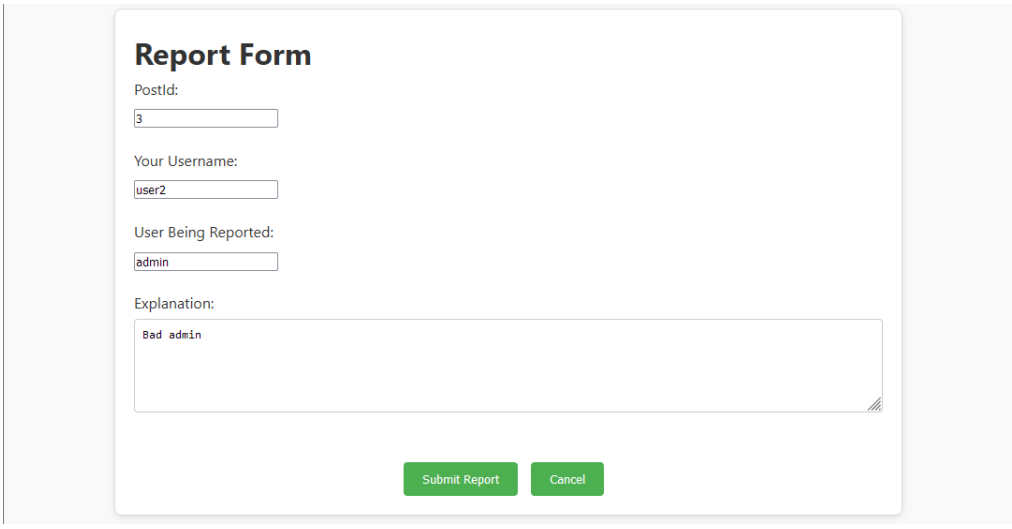
Using this new user, we will view the community guidelines and terms of service that the Admin added.



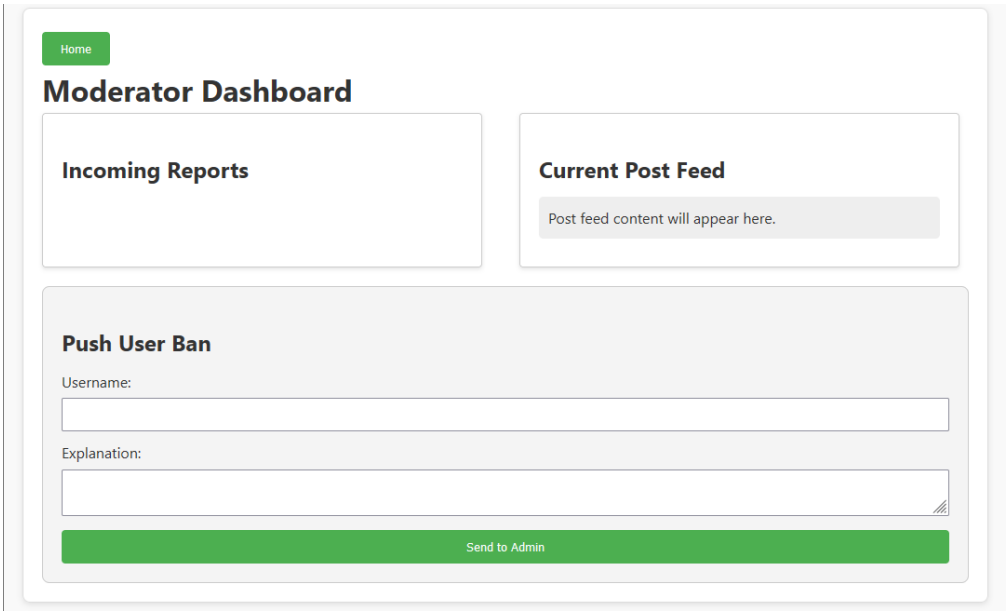


I will then report a user as a user (did not work during presentation because the user who created the post got deleted)

In an event a user/artist needs to report another user then they can simply file a report on the report form page. Once submitted it will be reviewed my the moderator .



Next we will log in as a Mod



Whenever a User or Artist submits a report on the report page, it appears in the [Incoming Reports] section of the dashboard. There, I can see who filed the report, who is being reported, and their explanation. Users and Artists are not the only ones responsible for addressing violations of our guidelines; as a moderator, I can also review the [current post feed] and delete any post that breaches our site's policies. Furthermore, if a report in the [Incoming Reports] section raises questions about a potential ban, I will file my own report in the [Push User Ban] section. Any reports submitted there are sent to the Admin, who will review them and implement any necessary bans.

There was no artist implementation to demonstrate.